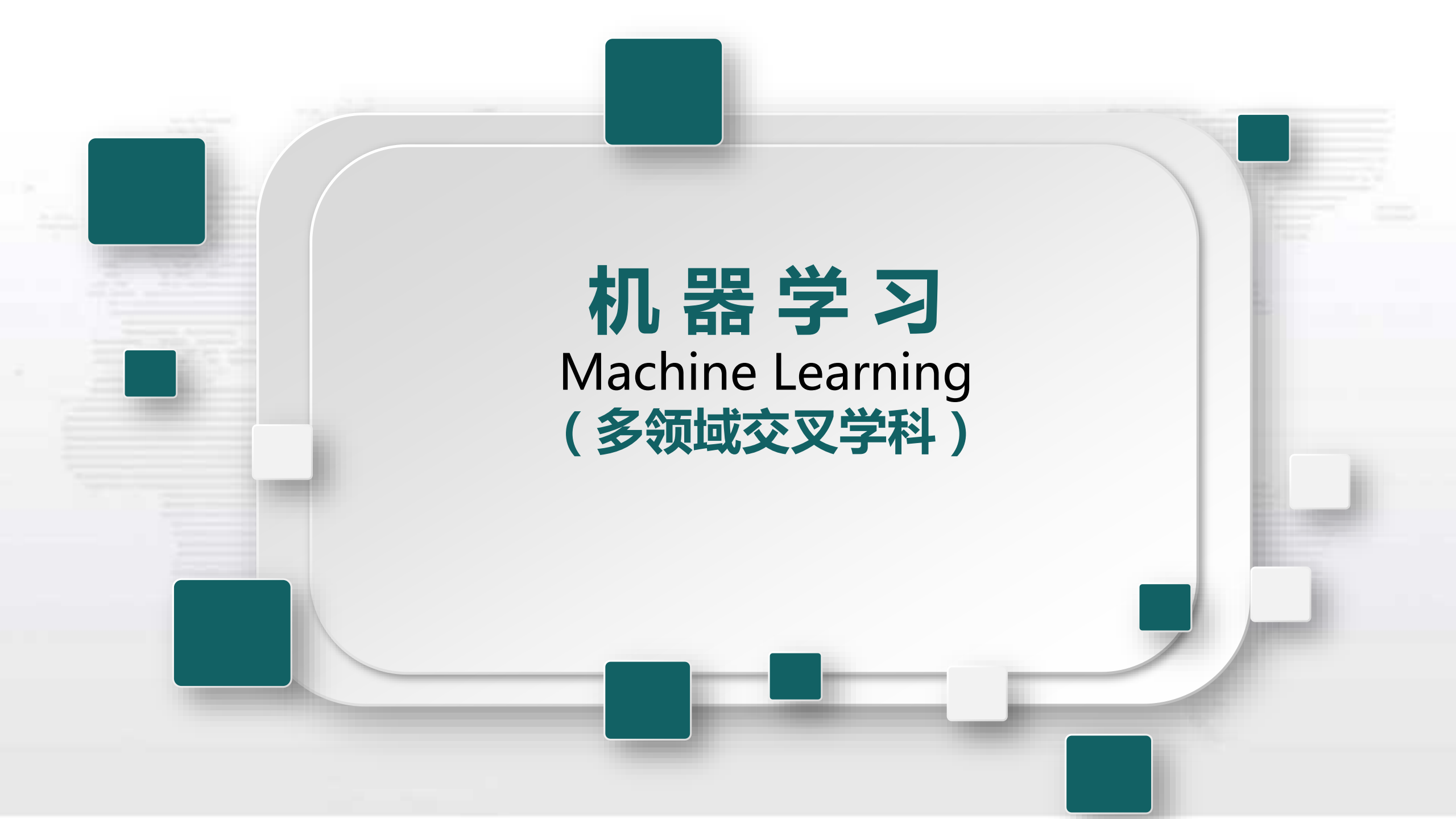




机器学习

Machine Learning
(多领域交叉学科)

1

梯度下降法

Gradient Descent

- 不是一个机器学习算法
- 是一种基于搜索的最优化方法
- 作用：最小化一个损失函数
- 梯度上升法：最大化一个效用函数

优化一个目标函数

最终求解的线性回归的模型，本质就是最小化一个损失函数，上一节直接计算出的最小化损失函数对应的参数的数学解，但实际上很多机器学习的模型是计算不出这种数学解的

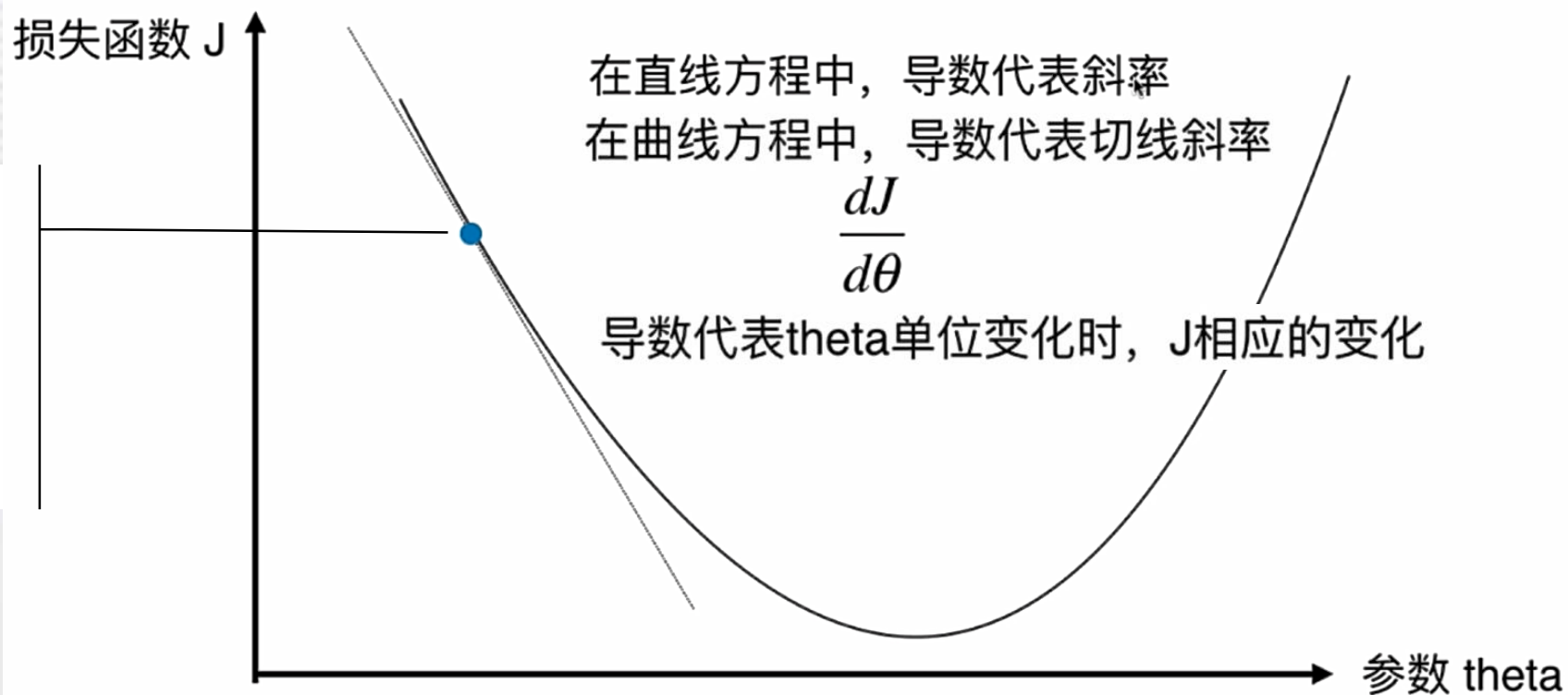
针对这样的模型，要基于搜索的策略，来找到最优解，那么梯度下降法就是在机器学习领域最小化损失函数的一个最为常用的方法

1

梯度下降法

总体上说，用梯度法求一个目标函数的最优值，是一种非常重要的技术手段。

导数为负值，即切线斜率为负值，所以当 θ 增加时， J 是减少的，换句话说，导数可以代表方向，对应 J 增大的方向，这里是 X 轴的负方向，若求 J 的最小值，就需要向导数的负方向移动

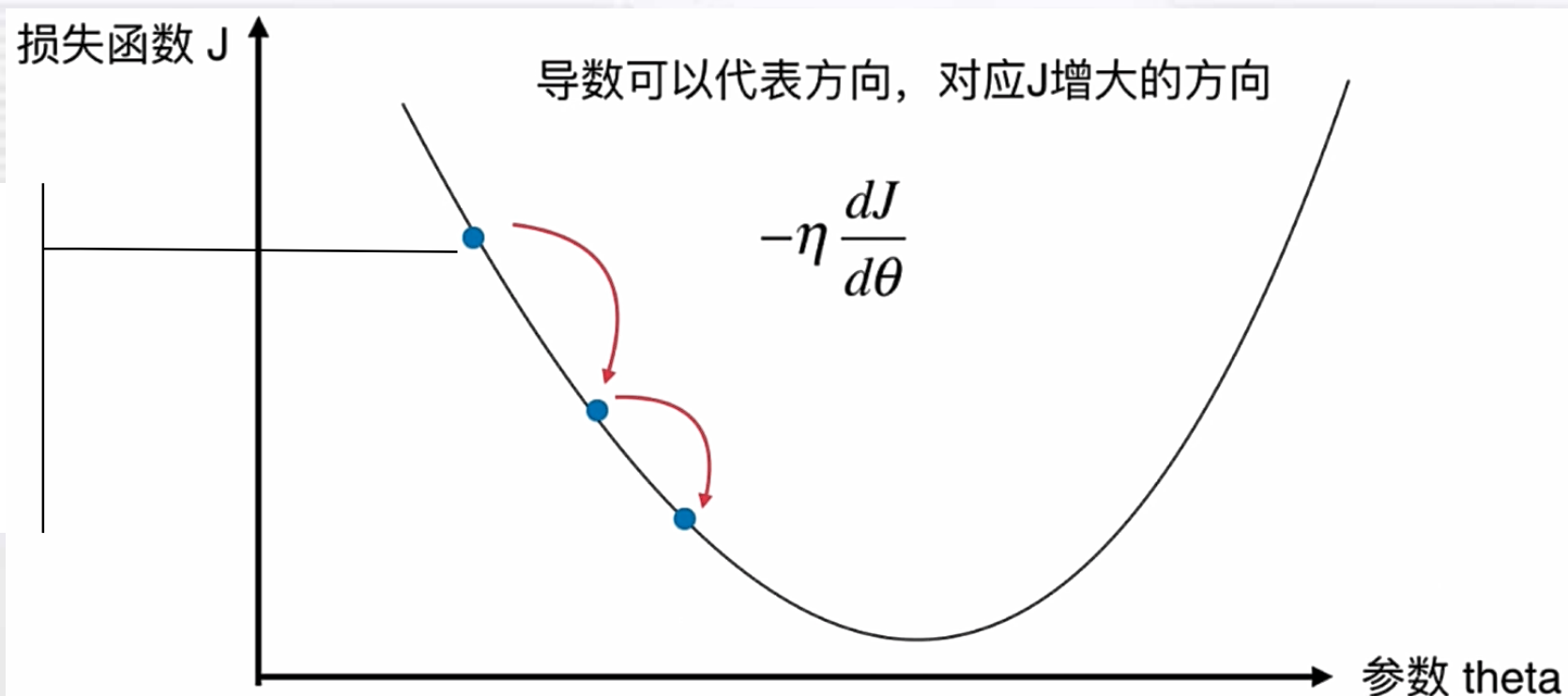


1

梯度下降法

总体上说，用梯度法求一个目标函数的最优值，是一种非常重要的技术手段。

对于一个点来说，求出它的导数，就可以知道这个点对应的J增大的方向，若求最小值，需要向导数减小的方向移动，即：
导数 $\times (-1)$ ，移动的步长为 η 伊塔



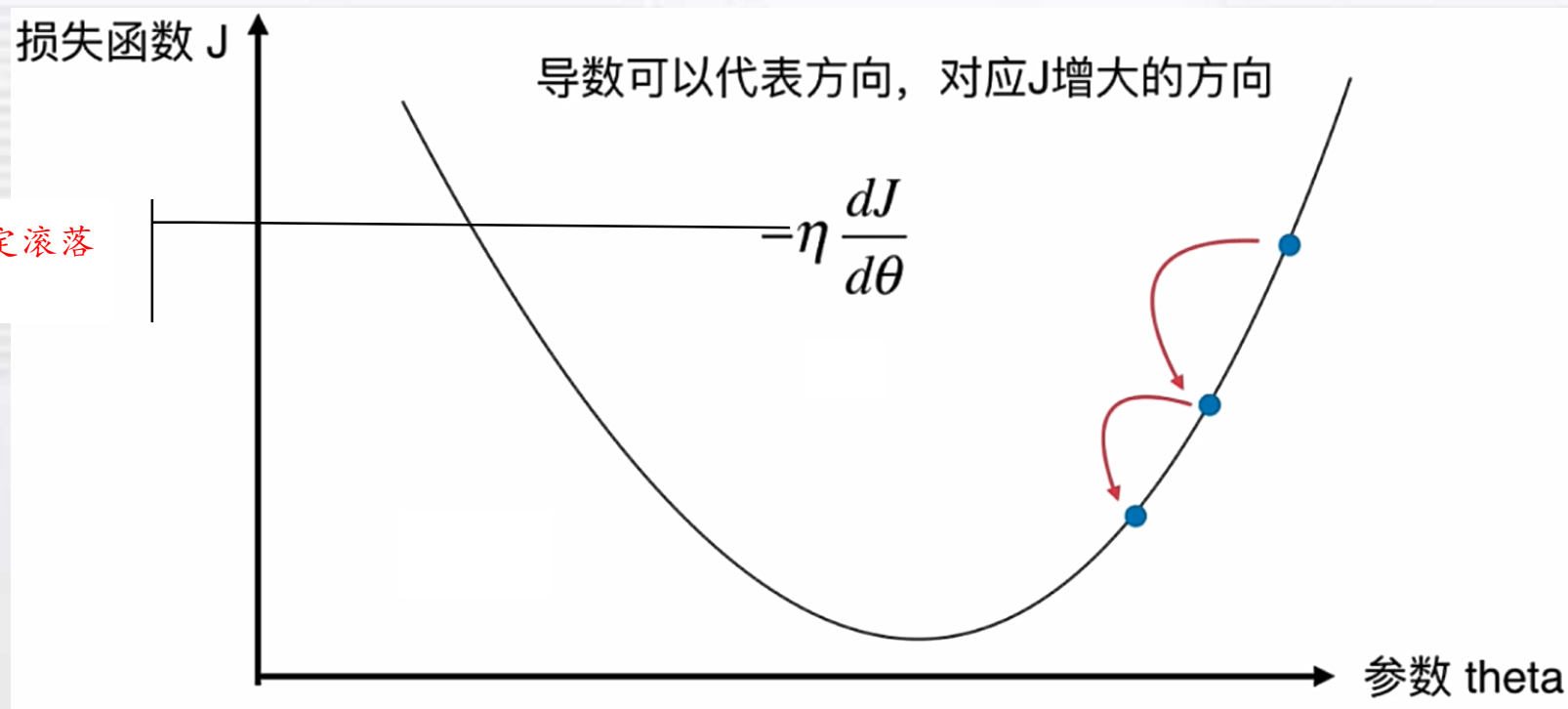
这个点是随着损失函数的降低，而逐步下降的，所以叫梯度下降法。
一维函数中，求导；多维函数中，对各个方向的分量分别求导，最终得到的方向就是梯度。

1

梯度下降法

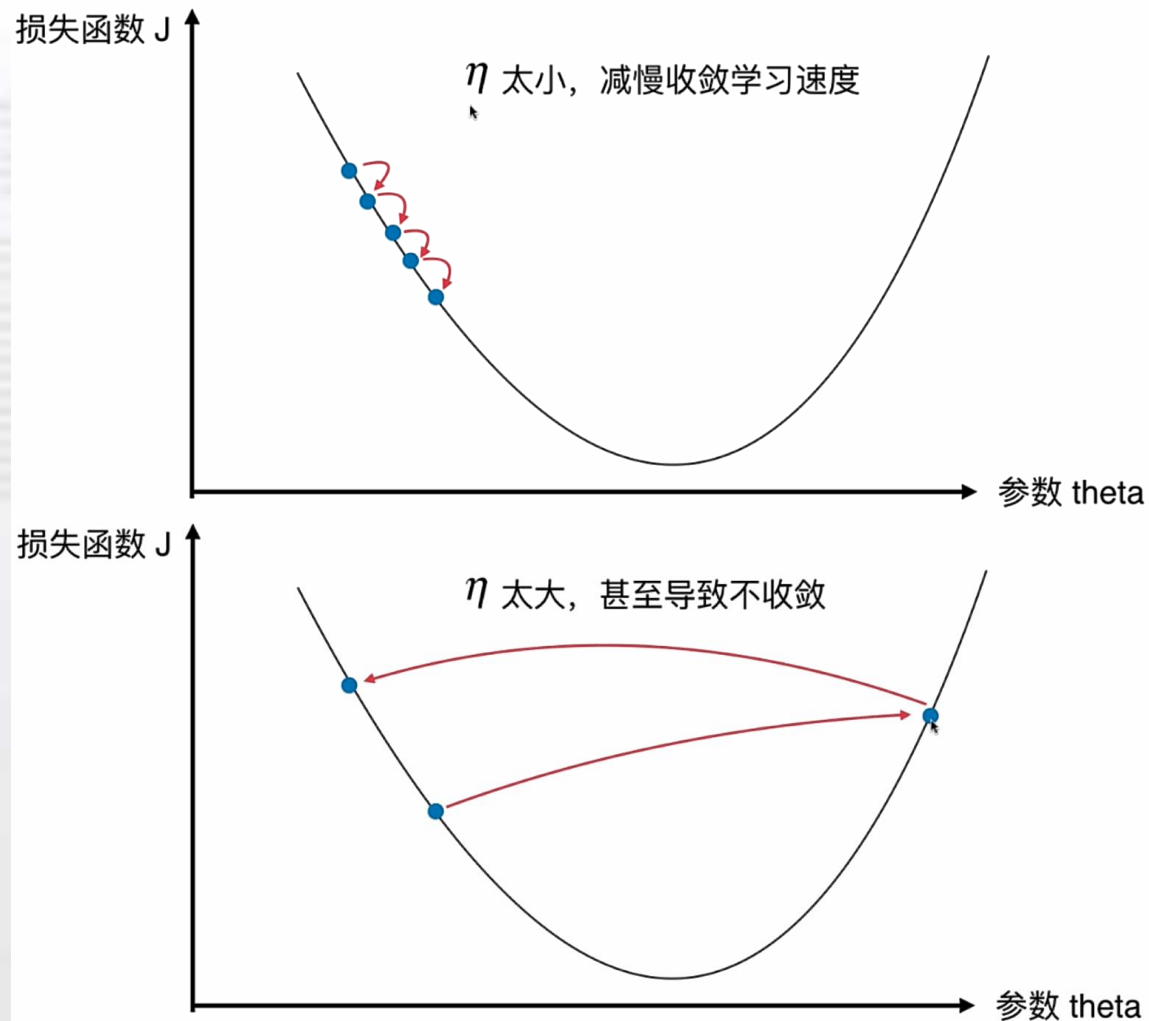
总体上说，用梯度法求一个目标函数的最优值，是一种非常重要的技术手段。

移动的步长为 η ：决定滚落的速度



1

梯度下降法



- η 称为学习率(learning rate)
- η 的取值影响获得最优解的速度
- η 取值不合适, 甚至得不到最优解
- η 是梯度下降法的一个超参数

梯度下降法主要为了用于更复杂的函数
对于二次曲线函数, 直接求出数学解, 但是很多时候
会遇到非常复杂的高维函数

1

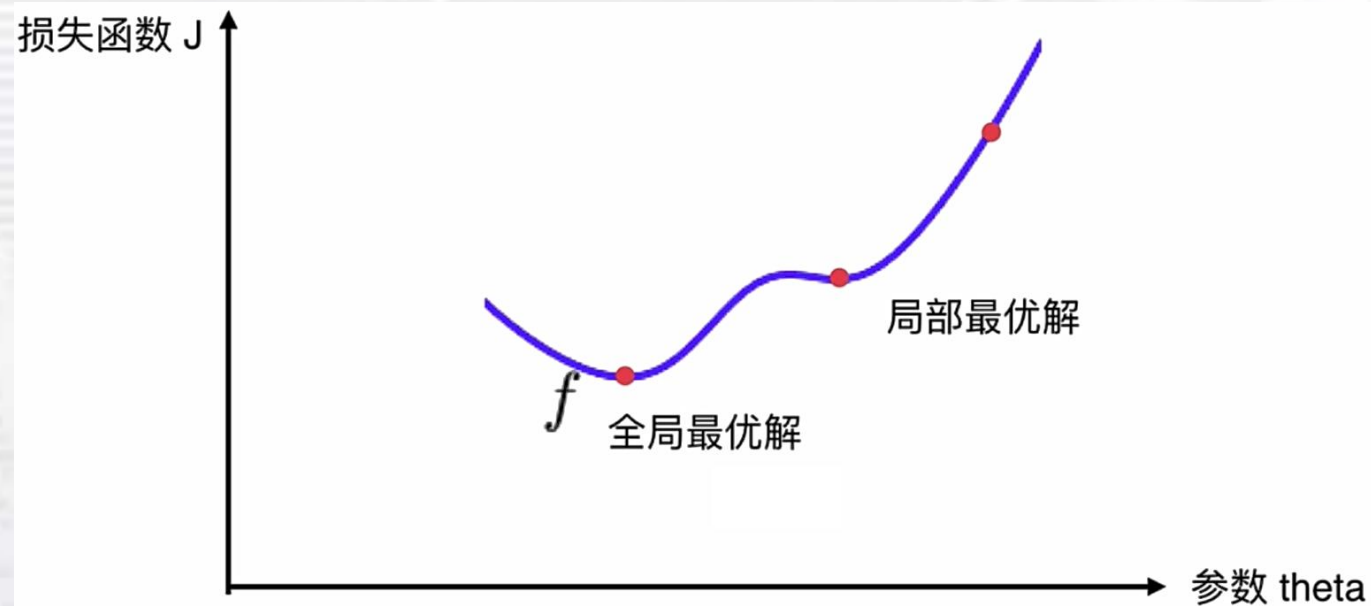
梯度下降法

- 并不是所有函数都有唯一的极值点

梯度下降法主要为了用于更复杂的函数
对于二次曲线函数，直接求出数学解，但是很多时候会遇到非常复杂的高维函数

解决方案：

- 多次运行，随机化初始点
- 梯度下降法的初始点也是一个超参数



1

梯度下降法

线性回归中使用梯度下降法

目标：使 $\sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$ 尽可能小

线性回归法的损失函数具有唯一的最优解

1

梯度下降法

梯度下降法原理-程序模拟

```
In [1]: 1 import numpy as np
        2 import matplotlib.pyplot as plt

In [2]: 1 plot_x = np.linspace(-1, 6, 141)
        2 plot_x
```

画出损失函数的曲线，取X轴（-1，6），
为了使曲线平滑，等间距采140个点

```
Out[2]: array([-1.   , -0.95, -0.9  , -0.85, -0.8  , -0.75, -0.7  , -0.65, -0.6  ,
               -0.55, -0.5  , -0.45, -0.4  , -0.35, -0.3  , -0.25, -0.2  , -0.15,
               -0.1  , -0.05,  0.   ,  0.05,  0.1  ,  0.15,  0.2  ,  0.25,  0.3  ,
                0.35,  0.4  ,  0.45,  0.5  ,  0.55,  0.6  ,  0.65,  0.7  ,  0.75,
                0.8  ,  0.85,  0.9  ,  0.95,  1.   ,  1.05,  1.1  ,  1.15,  1.2  ,
                1.25,  1.3  ,  1.35,  1.4  ,  1.45,  1.5  ,  1.55,  1.6  ,  1.65,
                1.7  ,  1.75,  1.8  ,  1.85,  1.9  ,  1.95,  2.   ,  2.05,  2.1  ,
                2.15,  2.2  ,  2.25,  2.3  ,  2.35,  2.4  ,  2.45,  2.5  ,  2.55,
                2.6  ,  2.65,  2.7  ,  2.75,  2.8  ,  2.85,  2.9  ,  2.95,  3.   ,
                3.05,  3.1  ,  3.15,  3.2  ,  3.25,  3.3  ,  3.35,  3.4  ,  3.45,
                3.5  ,  3.55,  3.6  ,  3.65,  3.7  ,  3.75,  3.8  ,  3.85,  3.9  ,
                3.95,  4.   ,  4.05,  4.1  ,  4.15,  4.2  ,  4.25,  4.3  ,  4.35,
                4.4  ,  4.45,  4.5  ,  4.55,  4.6  ,  4.65,  4.7  ,  4.75,  4.8  ,
                4.85,  4.9  ,  4.95,  5.   ,  5.05,  5.1  ,  5.15,  5.2  ,  5.25,
                5.3  ,  5.35,  5.4  ,  5.45,  5.5  ,  5.55,  5.6  ,  5.65,  5.7  ,
                5.75,  5.8  ,  5.85,  5.9  ,  5.95,  6.   ])
```

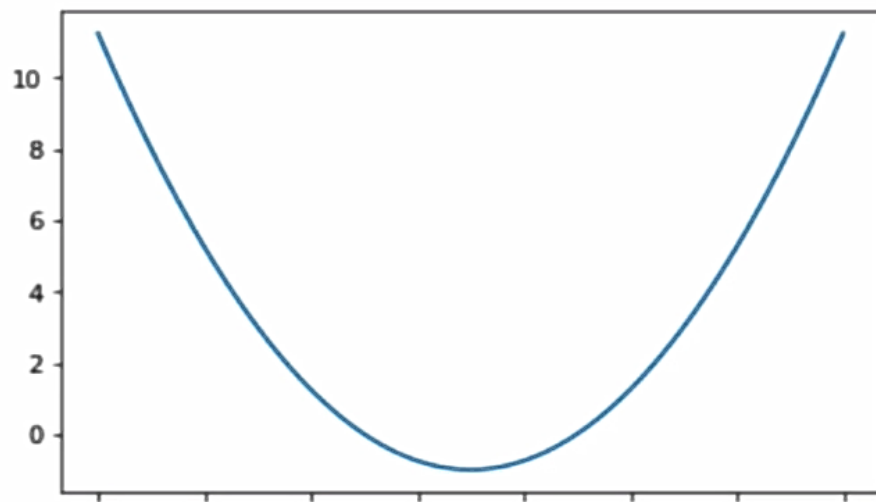
1

梯度下降法

梯度下降法模拟

```
In [3]: 1 plot_y = (plot_x-2.5)**2-1
```

```
In [4]: 1 plt.plot(plot_x, plot_y)
2 plt.show()
```



```
In [5]: 1 def dJ(theta):
2         return 2*(theta-2.5)
```

```
In [6]: 1 def J(theta):
2         return (theta-2.5)**2-1
```

```
In [7]: 1 eta = 0.1
2 epsilon = 1e-8
3
4 theta = 0.0
5 while True:
6     gradient = dJ(theta)
7     last_theta = theta
8     theta = theta - eta * gradient
9
10    if(abs(J(theta) - J(last_theta)) < epsilon):
11        break
12
13 print(theta)
14 print(J(theta))
```

2.499891109642585
-0.999999998814289

梯度下降法的验证

定义求导函数 和 Theta对应的损失函数

1

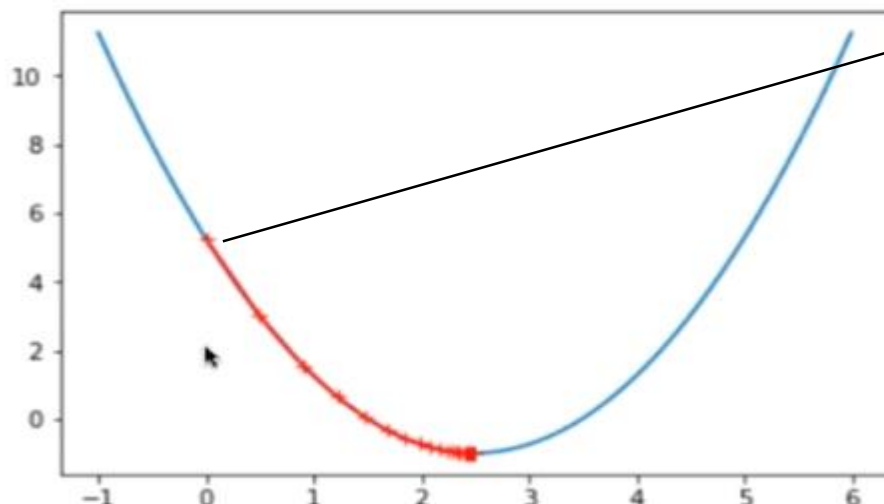
梯度下降法

```
In [ ]: 1 theta = 0.0
2 theta_history = [theta]
3 while True:
4     gradient = dJ(theta)
5     last_theta = theta
6     theta = theta - eta * gradient
7     theta_history.append(theta)
8
9     if(abs(J(theta) - J(last_theta)) < epsilon):
10         break
11
12 plt.plot(plot_x, J(plot_x))
13 plt.plot(np.array(theta_history), J(np.array(theta_history)), color='r', marker='+')
14 plt.show()
```

```
In [9]: 1 len(theta_history)
```

```
Out[9]: 46
```

表明经过45次查找



Theta=0, 开始时向下移动幅度比较大, 因为曲线陡 (gradient值大), 后面曲线平缓 (gradient值小)

1

梯度下降法

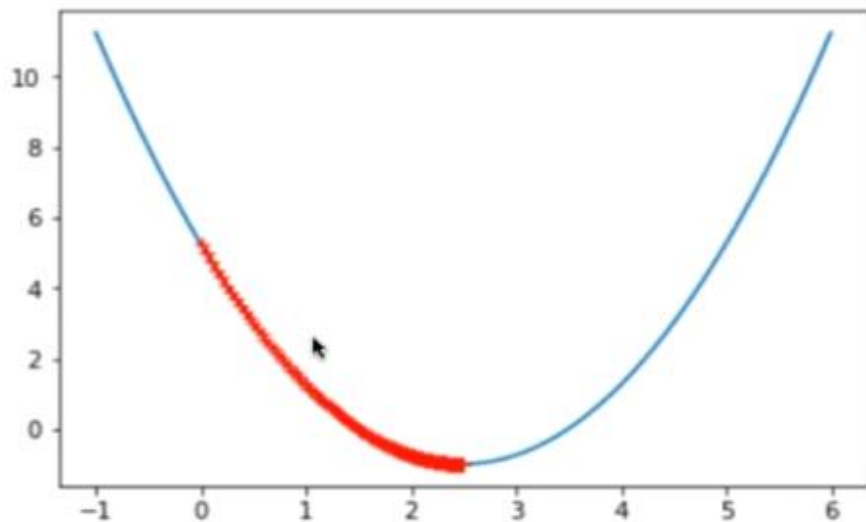
再次对梯度下降进行封装

```
In [10]: 1 def gradient_descent(initial_theta, eta, epsilon=1e-8):
2         theta = initial_theta
3         theta_history.append(initial_theta)
4
5         while True:
6             gradient = dJ(theta)
7             last_theta = theta
8             theta = theta - eta * gradient
9             theta_history.append(theta)
10
11             if(abs(J(theta) - J(last_theta)) < epsilon):
12                 break
13
14 def plot_theta_history():
15     plt.plot(plot_x, J(plot_x))
16     plt.plot(np.array(theta_history), J(np.array(theta_history)), color
17             plt.show()
```

1

梯度下降法

```
In [11]: 1 eta = 0.01  
2 theta_history = []  
3 gradient_descent(0., eta)  
4 plot_theta_history()
```

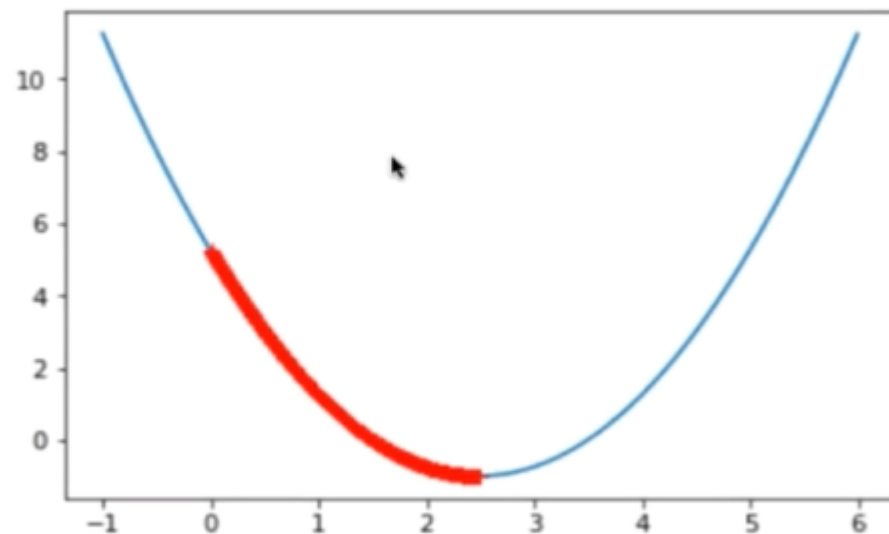


```
In [12]: 1 len(theta_history)
```

Out[12]: 424

表明经过424次查找

```
In [13]: 1 eta = 0.001  
2 theta_history = []  
3 gradient_descent(0, eta)  
4 plot_theta_history()
```



```
In [14]: 1 len(theta_history)
```

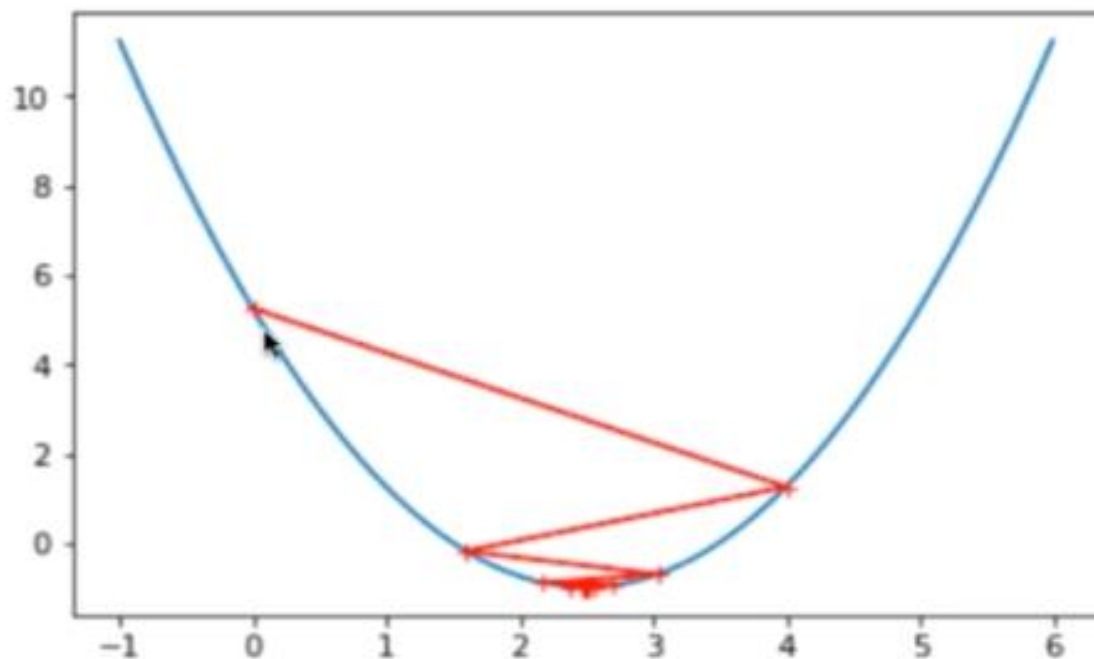
Out[14]: 3682

1

梯度下降法

In [16]:

```
1 eta = 0.8  
2 theta_history = []  
3 gradient_descent(0, eta)  
4 plot_theta_history()
```



1

梯度下降法

```
In [18]: 1 eta = 1.1
          2 theta_history = []
          3 gradient_descent(0, eta)

--
OverflowError                                Traceback (most recent call last)
<ipython-input-18-81ad09e9d3e0> in <module>()
      1 eta = 1.1
      2 theta_history = []
----> 3 gradient_descent(0, eta)

<ipython-input-10-781cd861e2c3> in gradient_descent(initial_theta, eta, epsilon)
      9         theta_history.append(theta)
     10
----> 11         if(abs(J(theta) - J(last_theta)) < epsilon):
     12             break
     13

<ipython-input-6-ae1577092099> in J(theta)
      1 def J(theta):
----> 2         return (theta-2.5)**2-1

OverflowError: (34, 'Result too large')
```

eta为1.1，导致梯度值越来越大

1

梯度下降法

```
In [19]: 1 def J(theta):  
2     try:  
3         return (theta-2.5)**2 - 1.  
4     except:  
5         return float('inf')
```

抛出异常，若超出范围，
则返回浮点数的最大值。

```
In [*]: 1 eta = 1.1  
2 theta_history = []  
3 gradient_descent(0, eta)
```

```
In [10]: 1 def gradient_descent(initial_theta, eta, epsilon=1e-8):  
2     theta = initial_theta  
3     theta_history.append(initial_theta)  
4  
5     while True:  
6         gradient = dJ(theta)  
7         last_theta = theta  
8         theta = theta - eta * gradient  
9         theta_history.append(theta)  
10  
11         if(abs(J(theta) - J(last_theta)) < epsilon):  
12             break
```

仍旧执行此程序，会出现死循环，这是因为：
while True:

.....
if(abs(J(theta) - J(last_theta)) < epsilon)
其中：J(theta)增大到无穷时，无穷-无穷
=not a number，即NAN，因此if条件永远
不会触发

1

梯度下降法

```
In [21]: 1 def gradient_descent(initial_theta, eta, n_iters = 1e4, epsilon=1e-8):
2
3     theta = initial_theta
4     theta_history.append(initial_theta)
5     i_iter = 0
6
7     while i_iter < n_iters:
8         gradient = dJ(theta)
9         last_theta = theta
10        theta = theta - eta * gradient
11        theta_history.append(theta)
12
13        if(abs(J(theta) - J(last_theta)) < epsilon):
14            break
15
16        i_iter += 1
```

调整循环次数。

```
In [22]: 1 eta = 1.1
2 theta_history = []
3 gradient_descent(0, eta)
```

```
In [23]: 1 len(theta_history)
```

Out[23]: 10001

```
In [24]: 1 theta_history[-1]
```

Out[24]: nan

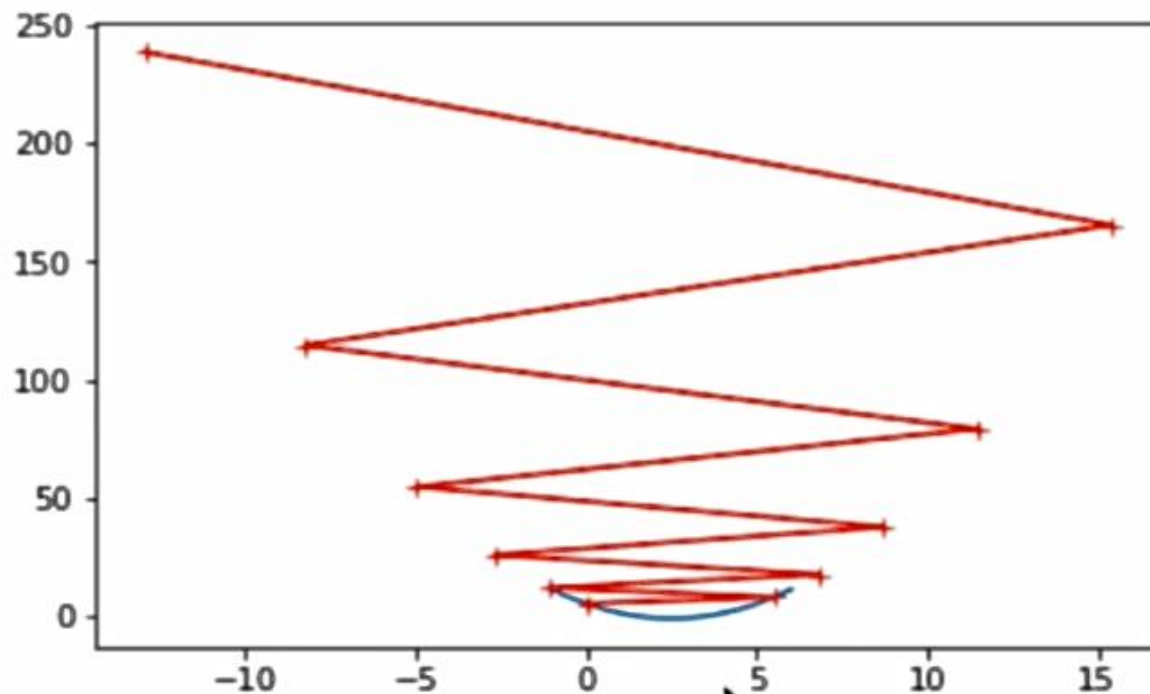
1

梯度下降法

eta可默认设置为0.01，对于大多数函数来说是一个保险的数值。

In [25]:

```
1 eta = 1.1
2 theta_history = []
3 gradient_descent(0, eta, n_iters=10)
4 plot_theta_history()
```





谢谢